

Workflow: Cloudflare + D1 Deployment for POS App

Objective

Deploy the POS app to Cloudflare Worker and connect it to Cloudflare D1 so the app can load and save real data instead of relying only on browser local storage.

Phase 1: Cloudflare setup

1. Open terminal in the project folder. 2. Log in to Cloudflare: `npx wrangler login` 3. Create the D1 database: `npx wrangler d1 create pos-thai-db` 4. Put the generated `database_id` into `wrangler.toml`. 5. Apply the schema to the live database: `npx wrangler d1 execute pos-thai-db --remote --file=./schema.sql`

Phase 2: Worker setup

1. Create a Worker entry file named `worker.js`. 2. Set the Worker to serve `/` for the frontend, `/api/health` for health checks, and `/api/state` for app data storage and retrieval. 3. Bind D1 to the Worker. 4. Deploy: `npx wrangler deploy`

Phase 3: State sync flow

1. On first app load, the frontend calls `GET /api/state`. 2. If the state is empty, the Worker seeds default POS data. 3. The app loads menu, categories, ingredients, and orders from the API response. 4. After any UI change, the app sends `POST /api/state`. 5. The Worker saves the JSON state back to D1.

Phase 4: Verification

Run these checks after deployment: - `curl -sS https://pos-thai-app.satesawa.workers.dev/api/health` - `curl -sS https://pos-thai-app.satesawa.workers.dev/api/state` Expected result: `/api/health` returns `ok: true` and `/api/state` returns valid JSON with POS data.

Phase 5: Future edits

When changing UI or business logic: 1. Edit the relevant file in the project. 2. Validate syntax. 3. Deploy with Wrangler. 4. Reload the live URL and verify the API still responds.

Files in project

`index.html`, `styles.css`, `worker.js`, `wrangler.toml`, `schema.sql`

Production URL

`https://pos-thai-app.satesawa.workers.dev`

Recommended next improvements

- Add separate API endpoints for products, orders, stock, and expenses - Add auth for admin users - Add backup/restore for data export - Add automatic seed validation and error logging